11. Keylime Tenant 관리 도구 및 CLI 분석

11.1 개요

Keylime Tenant는 Keylime 시스템의 핵심 관리 도구로, 에이전트 프로비저닝, 정책 관리, 그리고 암호화된 페이로드 전송을 담당하는 명령줄 인터페이스입니다. `keylime/tenant.py` 의 `Tenant` 클래스를 중심으로 구현되어 있으며, 관리자가 원격 시스템의 무결성을 검증하고 보안 정책을 적용할 수 있는 포괄적인 기능을 제공합니다.

11.2 Tenant 클래스 핵심 구조

11.2.1 클래스 정의 및 주요 속성

```
class Tenant:

"""Simple command processor example."""

# 네트워크 연결 설정

verifier_ip: Optional[str] = None

verifier_port: Optional[str] = None

agent_ip: Optional[str] = None

agent_port: Optional[str] = None

registrar_ip: Optional[str] = None

registrar_port: Optional[str] = None

# 암호화 키 관리

K: bytes = b""  # 대칭 암호화 키
```

```
V: bytes = b""  # 검증 키

U: bytes = b""  # XOR 결합 키 (K ⊕ V)

auth_tag = None # HMAC 인증 태그

# 정책 관리

tpm_policy = None

runtime_policy: str = ""

mb_policy = None

# TLS 보안 설정

tls_context: Optional[ssl.SSLContext] = None

agent_tls_context: Optional[ssl.SSLContext] = None
```

11.2.2 초기화 및 TLS 설정

`__init__()` 메서드에서는 다음과 같은 핵심 초기화 작업을 수행합니다:

```
def  __init__(self) -> None:

# 설정 파일에서 기본값 로드

self.verifier_ip = config.get("tenant", "verifier_ip")

self.verifier_port = config.get("tenant", "verifier_port")

self.registrar_ip = config.get("tenant", "registrar_ip")

self.registrar_port = config.get("tenant", "registrar_port")

# mTLS 설정

self.enable_agent_mtls = config.getboolean("tenant", "enable_agent_mtls")

# TLS 컨텍스트 생성
```

```
(cert, key, trusted_ca, key_password), verify_server_cert = web_util.get_tls_opt
"tenant", is_client=True, logger=logger
)
if  all([(cert and os.path.isfile(cert)), (key and os.path.isfile(key))]):
self.tls_context = web_util.generate_tls_context(
cert, key, trusted_ca, key_password, verify_server_cert,
is_client=True, logger=logger
)
```

11.3 에이전트 프로비저닝 및 관리

11.3.1 에이전트 추가 프로세스 (`init_add`)

에이전트 추가는 다음 단계로 진행됩니다:

1. **Registrar에서 에이전트 데이터 조회**:

```
self.registrar_data = registrar_client.getData(
self.registrar_ip, self.registrar_port, self.agent_uuid, self.tls_context
)
```

2. **API 버전 자동 감지**:

```
with RequestsClient(
f"{bracketize_ipv6(self.agent_ip)}:{self.agent_port}",
tls_enabled=self.enable_agent_mtls,
```

```
tls_context=tls_context,
) as get_version:
res = get_version.get("/version")
if res and res.status_code == 200:
data = res.json()
api_version = data["results"]["supported_version"]
```

3. **정책 처리 및 암호화 키 설정**:

```
(
self.tmp_policy,
self.mb_policy,
self.mb_policy_name,
self.runtime_policy_name,
self.ima_sign_verification_keys,
self.runtime_policy,
self.runtime_policy_key,
) = policies.process_policy(cast(policies.ArgsType, args))
```

11.3.2 TPM Quote 검증 (`validate_tpm_quote`)

TPM Quote 검증은 에이전트의 신뢰성을 확인하는 핵심 과정입니다:

```
def  validate_tpm_quote(self, public_key: str, quote: str, hash_alg: algorithms.
failure = self.tpm_instance.check_quote(
```

```
AgentAttestState(self.agent_uuid),
self.nonce,
public_key,
quote,
self.registrar_data["aik_tpm"],
hash_alg=hash_alg,
compressed=(self.supported_version == "1.0"),
)
if failure:
return False
# EK 인증서 검증
if not self.check_ek(self.registrar_data["ekcert"]):
return False
```

11.3.3 Verifier에 에이전트 등록 (do_cvadd)

```
def  do_cvadd(self) -> None:
data = {
"v": base64.b64encode(self.V).decode("utf-8") if  self.V else None,
"cloudagent_ip": agent_ip,
"cloudagent_port": agent_port,
"tpm_policy": json.dumps(self.tpm_policy),
```

```
"runtime_policy": self.runtime_policy,

"mb_policy": self.mb_policy,

"ak_tpm": self.registrar_data["aik_tpm"],

"supported_version": self.supported_version,

}

do_cv = RequestsClient(self.verifier_base_url, True, tls_context=self.tls_contex

response = do_cv.post(

f"/v{self.api_version}/agents/{self.agent_uuid}",

data=json.dumps(data),

timeout=self.request_timeout

)
```

11.4 명령줄 옵션 처리 (`keylime/cli/options.py`)

11.4.1 옵션 검증 함수

`get_opts_error()` 함수는 명령줄 인수의 유효성을 검사합니다:

```
def  get_opts_error(args: Any) -> Tuple[bool, str]:

# 런타임 정책 필수 검증

if args.command in ["addallowlist", "addruntimepolicy"] and not (

args.allowlist or args.allowlist_url or args.runtime_policy or args.runtime_poli

):

return True, "--allowlist or --runtime_policy is required to add a runtime polic
```

```
# 상호 배타적 옵션 검증

if args.allowlist and args.allowlist_url:

return True, "--allowlist and --allowlist-url cannot be specified at the same ti

# 체크섬 검증 요구사항

if args.runtime_policy_url and not args.runtime_policy_checksum:

return True, "--runtime_policy-url must have --runtime_policy-checksum to verifi
```

11.4.2 패스워드 추출 유틸리티

```
def  extract_password(pwstring: str) -> str:

# 파일 경로인지 확인

pwstring = os.path.expanduser(pwstring)

if os.path.exists(pwstring):

with  open(pwstring, encoding="utf-8") as fp:

pwstring = fp.read().strip()

# 환경 변수인지 확인

if pwstring in os.environ:

pwstring = os.environ[pwstring]

return pwstring
```

11.5 정책 처리 (keylime/cli/policies.py)

11.5.1 정책 처리 메인 함수

process_policy() 함수는 다양한 정책 유형을 통합 처리합니다:

```
def  process_policy(args: ArgsType) -> Tuple[Dict[str, Any], Optional[str], str,

# TPM 정책 파싱

tpm_policy = tpm_util.readPolicy(tmp_policy_str)

# IMA 서명 검증 키 처리

if  len(args["ima_sign_verification_keys"]) > 0:

tpm_policy["mask"] = hex(int(tpm_policy["mask"], 0) | (1 << config.IMA_PCR))

tenant_keyring = file_signatures.ImaKeyring()

for filename in args["ima_sign_verification_keys"]:

pubkey, keyidv2 = file_signatures.get_pubkey_from_file(filename)

tenant_keyring.add_pubkey(pubkey, keyidv2)

ima_sign_verification_keys = tenant_keyring.to_string()
```

11.5.2 레거시 정책 변환

```
# 레거시 allowlist를 새로운 런타임 정책으로 변환

if  "allowlist" in args and args["allowlist"] is not None:

logger.warning("WARNING: --allowlist is deprecated.")

runtime_policy_dict = convert_runtime_policy.convert_legacy_allowlist(args["allo

runtime_policy_raw = json.dumps(

convert_runtime_policy.update_runtime_policy(runtime_policy_dict, excl_path)

).encode()
```

```
runtime_policy = base64.b64encode(runtime_policy_raw).decode()
```

11.5.3 Measured Boot 정책 처리

```
# Measured Boot 정책 로드
if "mb_policy" in args and args["mb_policy"] is not None:
enforce_pcrs(tpm_policy, config.MEASUREDBOOT_PCRS, "measured boot")
# PCR 마스크 자동 활성화
for _pcr in config.MEASUREDBOOT_PCRS:
tpm_policy["mask"] = hex(int(tmp_policy["mask"], 0) | (1 << _pcr))
mb_policy_data = mba.policy_load(args["mb_policy"])
```

11.6 데이터 암호화 유틸리티 (`keylime/cmd/user_data_encrypt.py`)

11.6.1 암호화 함수

`encrypt()` 함수는 페이로드 암호화의 핵심입니다:

```
def encrypt(contents: bytes) -> Dict[str, bytes]:
# 32바이트 랜덤 키 생성
k = crypto.generate_random_key(32) # 대칭 암호화 키
v = crypto.generate_random_key(32) # 검증 키
u = crypto.strbitxor(k, v) # XOR 결합 키
# AES 암호화 수행
```

```
ciphertext = crypto.encrypt(contents, k)

# 복호화 테스트로 무결성 검증

recovered = crypto.decrypt(ciphertext, k)

if recovered != contents:

raise Exception("Test decryption failed")

return {"u": u, "v": v, "k": k, "ciphertext": ciphertext}
```

11.6.2 키 파일 생성

```
def  main() -> None:

with  open(infile, "rb") as f:

contents = f.read()

ret = encrypt(contents)

# 키 파일 저장 (Base64 인코딩)

with  open("content_keys.txt", "w", encoding="utf-8") as f:

f.write(base64.b64encode(ret["k"]).decode("utf-8") + "\n")

f.write(base64.b64encode(ret["v"]).decode("utf-8") + "\n")

f.write(base64.b64encode(ret["u"]).decode("utf-8") + "\n")

# 암호화된 페이로드 저장

with  open("content_payload.txt", "w", encoding="utf-8") as f:

f.write(ret["ciphertext"].decode("utf-8"))
```

11.7 명령줄 인터페이스 구조

11.7.1 주요 명령어

Tenant CLI는 다음과 같은 주요 명령어를 제공합니다:

```
# 에이전트 관리
parser.add_argument("-c", "--command", choices=[
"add", # 에이전트 추가
"update", # 에이전트 업데이트
"delete", # 에이전트 삭제
"status", # 에이전트 상태 조회
"reactivate", # 에이전트 재활성화
])

# 정책 관리
parser.add_argument("--runtime-policy", help="런타임 정책 파일 경로")
parser.add_argument("--mb-policy", help="Measured Boot 정책")
parser.add_argument("--tpm_policy", help="TPM 정책 (JSON 형식)")

# 페이로드 전송
parser.add_argument("-f", "--file", help="평문 파일 전송")
parser.add_argument("-k", "--key", help="암호화 키 파일")
parser.add_argument("--cert", help="인증서 생성 및 전송")
```

11.7.2 네트워크 설정 옵션

```
# 대상 호스트 설정
parser.add_argument("-t", "--targethost", dest="agent_ip")
parser.add_argument("-tp", "--targetport", dest="agent_port")

# Verifier 설정
parser.add_argument("-v", "--cv", dest="verifier_ip")
parser.add_argument("-vp", "--cvport", dest="verifier_port")

# Registrar 설정
parser.add_argument("-r", "--registrarhost", dest="registrar_ip")
parser.add_argument("-rp", "--registrarport", dest="registrar_port")
```

11.8 암호화된 페이로드 전송 프로세스

11.8.1 파일 기반 페이로드 처리

```
if args["file"] is not None:
# 파일 내용 읽기
with  open(str(args["file"]), "rb") as f:
contents = f.read()
# 암호화 수행
```

```
ret = user_data_encrypt.encrypt(contents)

self.K = ret["k"] # 대칭 키

self.U = ret["u"] # XOR 키

self.V = ret["v"] # 검증 키

self.payload = ret["ciphertext"] # 암호화된 데이터
```

11.8.2 인증서 패키지 생성

```
if args["ca_dir"] is not None:

# CA 디렉토리에서 인증서 패키지 생성

cert_pkg, serial, subject = ca_util.cmd_certpkg(args["ca_dir"], self.agent_uuid)

# 철회 알림용 인증서 추가

rev_package, _, _ = ca_util.cmd_certpkg(args["ca_dir"], "RevocationNotifier")

# ZIP 파일로 패키징

sf = io.BytesIO(cert_pkg)

with zipfile.ZipFile(sf, "a", compression=zipfile.ZIP_STORED) as zf:

zf.writestr("RevocationNotifier-cert.crt", cert)

# 추가 파일 포함

if args["incl_dir"] is not None:

for filename in files:

with  open(os.path.join(incl_dir, filename), "rb") as f:

zf.writestr(os.path.basename(f.name), f.read())

# 인증서 패키지 암호화
```

```
ret = user_data_encrypt.encrypt(cert_pkg)

self.metadata = {"cert_serial": serial, "subject": subject}
```

11.8.3 HMAC 인증 태그 생성

```
def  preloop(self) -> None:

"""encrypt the agent UUID as a check for delivering the correct key"""

self.auth_tag = crypto.do_hmac(self.K, self.agent_uuid)

if config.INSECURE_DEBUG:

logger.debug("K: %s", base64.b64encode(self.K))

logger.debug("V: %s", base64.b64encode(self.V))

logger.debug("U: %s", base64.b64encode(self.U))

logger.debug("Auth Tag: %s", self.auth_tag)
```

11.9 보안 고려사항

11.9.1 mTLS 인증

```
# mTLS 비활성화 경고

if not self.enable_agent_mtls:

logger.warning(

"Warning: agent mTLS is currently disabled, keys will be sent in the clear! "

"This should only be used for testing."

)
```

```
# 에이전트별 인증서 검증
if self.registrar_data["mtls_cert"] != "disabled":
self.verify_custom = self.registrar_data["mtls_cert"]
self.agent_tls_context = web_util.generate_tls_context(
self.client_cert, self.client_key, self.trusted_server_ca,
self.client_key_password, self.verify_server_cert,
is_client=True, ca_cert_string=self.verify_custom, logger=logger,
)
```

11.9.2 페이로드 크기 제한

```
if self.payload is not None:
max_payload_size = config.getint("tenant", "max_payload_size")
if len(self.payload) > max_payload_size:
raise UserError(f"Payload size {len(self.payload)} exceeds max size {max_payload
```

11.10 명령 실행 흐름

11.10.1 메인 실행 로직

```
def main() -> None:
# 명령어별 분기 처리
```

```
if args.command == "add":

mytenant.init_add(vars(args))

if not mytenant.push_model:

mytenant.preloop() # HMAC 태그 생성

mytenant.do_quote() # TPM Quote 요청

mytenant.do_cvadd() # Verifier에 등록

if not mytenant.push_model and args.verify:

mytenant.do_verify() # 키 전달 확인

elif args.command == "update":

mytenant.init_add(vars(args))

mytenant.do_cvdelete(args.verifier_check) # 기존 등록 삭제

# ... 재등록 과정

elif args.command == "delete":

mytenant.do_cvdelete(args.verifier_check)

# 정책 관리 명령어들

elif args.command == "addruntimepolicy":

mytenant.do_add_runtime_policy(vars(args))
```

11.11 결론

Keylime Tenant는 TPM 기반 원격 증명 시스템의 핵심 관리 도구로서, 복잡한 암호화 프로토콜과 정책 관리를 사용자 친화적인 명령줄 인터페이스로 추상화합니다. `Tenant` 클래스의 체계적인 설계와 모듈화된 CLI 구조를 통해 안전하고 효율적인 에이전트 프로비저닝과 정책 관리를 제공하며, 암호화된 페이로드 전송을 통해 Zero Trust 보안 모델을 구현합니다.

주요 특징으로는 자동 API 버전 감지, 유연한 정책 처리, 강력한 암호화 메커니즘, 그리고 포괄적인 오류 처리가 있으며, 이를 통해 엔터프라이즈 환경에서의 대규모 배포와 관리를 지원합니다.